

LAYOUTLAB AR

A NOSS-Aligned Augmented Reality Trainer for Java Swing Interface Prototyping

Mohd Azlan bin Ab Aziz¹ | Hasnah binti Ngah²

Politeknik Mukah, Sarawak, Malaysia

Extended Abstract

Java Swing layout managers are difficult for novice learners because their rules remain invisible until code is compiled. LayoutLab AR is a browser-based augmented reality teaching aid aligned to IT-010-3:2016-C01, Application Prototype Development. Using an iPhone camera and an original tracked target, learners manipulate a virtual JFrame and complete three assessed BorderLayout missions. They place an application title in NORTH, test that CENTER absorbs remaining space during resizing, and diagnose why two buttons cannot independently occupy SOUTH before repairing the structure with a nested JPanel. Progress depends on touching the tracked 3D objects; conventional answer buttons cannot complete the assessment. The final AR state produces deterministic Java code showing the implemented prototype. The module integrates text, tracked 3D graphics, animation, bilingual audio, spatial interaction and generated code, exceeding the organiser's requirement for three media elements. Two formative evaluations of the wider LayoutLab browser platform (n=11 and n=6) established the learning need and informed the design; they are not claimed as proof of AR effectiveness. The competition build has completed production acceptance on an iPhone 13, including camera permission, stable tracking, all missions, audio and Java evidence. Seventy-five automated tests protect the engine, mission logic and code-generation path. LayoutLab AR therefore makes AR essential to constructing, testing and repairing an application prototype rather than using it as a decorative visual overlay.

Keywords: augmented reality, BorderLayout, Java Swing, NOSS, prototype development, TVET

Competition alignment	Application
NOSS	IT-010-3:2016 – Pembangunan Aplikasi, Tahap 3
Competency Unit	IT-010-3:2016-C01 – Application Prototype Development
Work Activities	Implement Application Prototype Mock-Up Flow; Conduct UI/UX Functionality Test

1. Problem and Objective

In BorderLayout, each region accepts one direct component and CENTER absorbs the remaining space. Learners commonly misread these rules, causing disappearing components and unpredictable resizing. The objective is to let learners construct, functionally test, diagnose and repair a Swing prototype through direct AR manipulation while producing auditable NOSS evidence.

2. AR Innovation and Assessed Flow

The camera recognises an original LayoutLab target and anchors a colour-coded 3D JFrame to it. Touch coordinates are raycast against mission-relevant meshes, making tracking, spatial anchoring and 3D interaction necessary for completion. BorderLayout is intentionally the assessed focus; FlowLayout and GridLayout remain enrichment content in the wider Playground.

Mission	AR task	Evidence
1 — Construct	Touch NORTH to place the application title.	Correct constraint and updated Java state.
2 — Test	Touch CENTER and observe the resize animation.	Verified dynamic layout behaviour.
3 — Diagnose and repair	Reveal the SOUTH collision and activate the JPanel repair.	Both buttons visible; score 3/3; nested-panel Java.

Why the AR is essential

The tracked object is the learning model, touch surface and assessment interface. The learner must act on the 3D regions to progress.

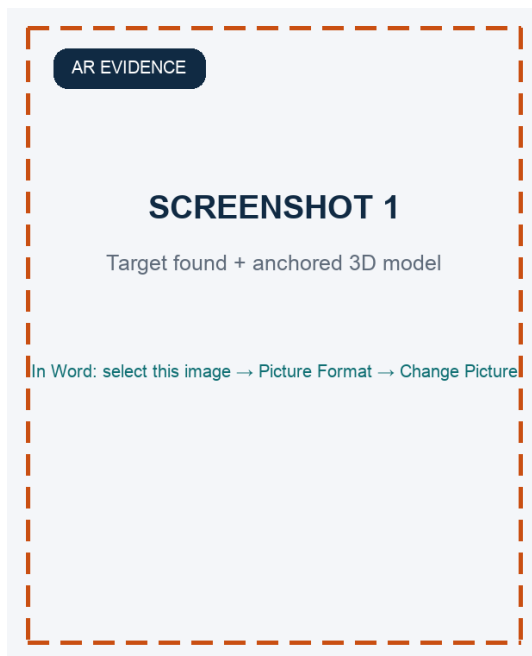


Figure 1. Target detected and 3D model anchored.

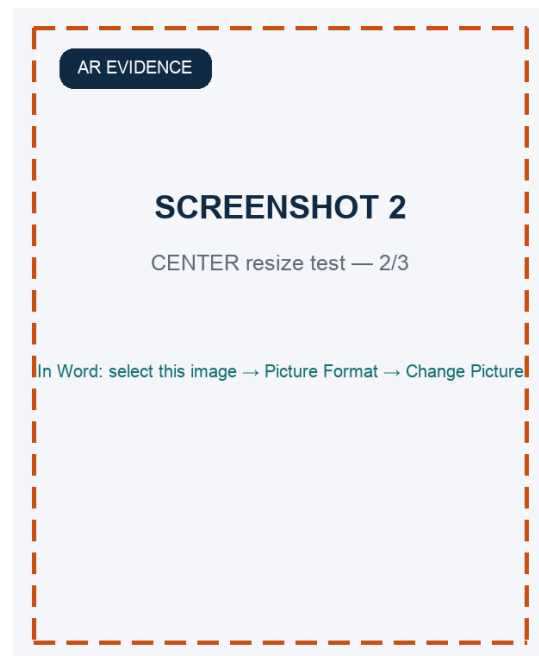


Figure 2. CENTER functionality test.

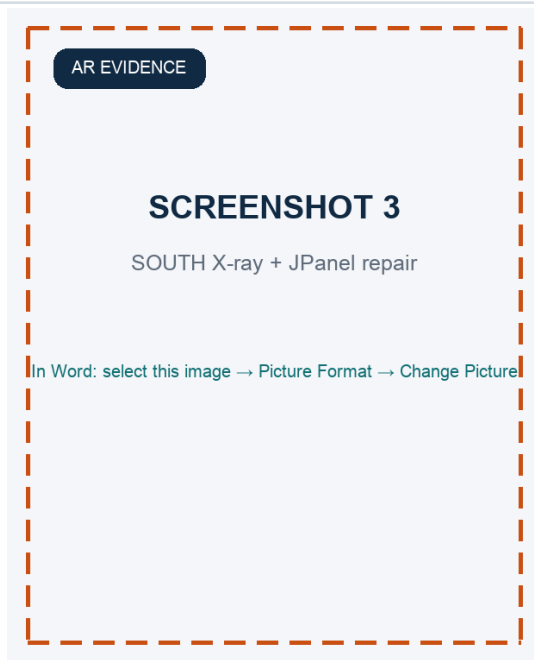


Figure 3. Collision diagnosis and repair.

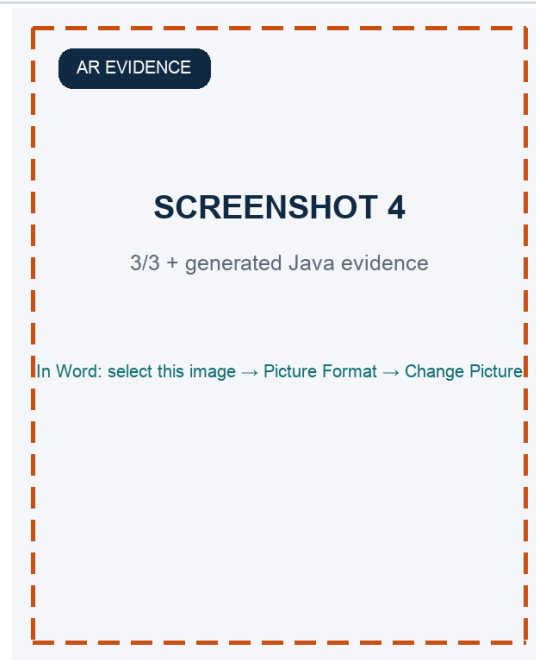


Figure 4. Completion and Java evidence.

3. Media Elements

Six elements have defined learning functions: text communicates missions and rules; tracked 3D graphics expose the layout model; animation demonstrates resize and collision behaviour; BM/English audio provides optional guidance; direct touch assesses spatial decisions; and deterministic Java connects the AR prototype to implementation evidence.

4. Validation and Competition Readiness

Evidence	Result	Interpretation
Formative platform evaluation	n=11 and n=6; visualisation, placement and resizing were valued.	Supports the need and design—not causal AR effectiveness.
Technical quality	75 automated tests and strict production build pass.	Supports deterministic, reproducible output.
iPhone 13 acceptance	HTTPS camera, tracking, direct touch, three missions, audio and Java verified.	Ready for live competition demonstration.

Before judging, the team should complete a small AR pilot measuring target-acquisition time, mission completion, wrong-region attempts and successful repair. This evidence should be reported separately from the earlier browser surveys.

5. Impact and Conclusion

LayoutLab AR shortens the distance between an abstract layout rule and its visible consequence. Learners construct a state, test behaviour, diagnose a failure, repair the structure and receive verifiable Java evidence. Lecturers gain a repeatable demonstrator aligned to a defined competency, while web deployment avoids native installation. The innovation is strongest when presented as a focused BorderLayout competency module: AR is the method of learning and assessment, and deterministic code—not generative AI—is the technical ground truth.

Declaration and Selected References

The work is original and its evidence can be verified. Generative AI does not determine correctness, scoring or assessed Java output.

Keuning, H., Jeuring, J., & Heeren, B. (2018). Automated feedback generation for programming exercises. ACM TOCE, 19(1).

Qian, Y., & Lehman, J. (2017). Difficulties in introductory programming: A literature review. *ACM TOCE*, 18(1).

Politeknik Mukah. (2026). LayoutLab surveys, AR acceptance records and technical documentation [Internal evidence pack].